

Enhancing SLP Vectorization through Hybrid Slice Selection

Chi-Hung Wei

Department of Computer Science and Information
Engineering
National Taiwan University
Taipei, Taiwan
r13922150@csie.ntu.edu.tw

Chih-Cheng Chen

Department of Computer Science and Information
Engineering
National Taiwan University
Taipei, Taiwan
r11922126@ntu.edu.tw

Shih-wei Liao*

Department of Computer Science and Information
Engineering
National Taiwan University
Taipei, Taiwan
liao@csie.ntu.edu.tw

Chi-Bang Kuan

MediaTek Inc.
Hsinchu, Taiwan
chibang.kuan@mediatek.com

Abstract

Superword-Level Parallelism (SLP) vectorization is a widely used compiler technique that transforms straight-line scalar code into efficient vector code by identifying and grouping independent, isomorphic instructions. Traditional SLP implementations typically initiate from store instructions and traverse upward along the def-use chain, making a single, local vectorization decision without being aware of the global knowledge. However, this heuristic strategy can be suboptimal due to the vectorization order, which can hinder performance and vectorization opportunities.

In this work, we present a hybrid slice selection algorithm that strategically combines dynamic programming with greedy heuristics. This approach enables more informed decisions by analyzing vectorization order and inter-tree dependences, leading to improved utilization of SIMD resources and better overall performance. Our experiments demonstrate significant cost reductions across several standard benchmark suites for both RISC-V and ARMv8 architectures. According to the LLVM cost model, the optimizations yield an additional reduction of up to 67% on RISC-V and 41% on ARMv8.

CCS Concepts

• **Software and its engineering** → **Compilers**; • **Theory of computation** → **Vector / streaming algorithms**; • **Computer systems organization** → **Single instruction, multiple data**.

Keywords

SLP, Auto Vectorization, LLVM, SIMD

ACM Reference Format:

Chi-Hung Wei, Shih-wei Liao, Chih-Cheng Chen, and Chi-Bang Kuan. 2025. Enhancing SLP Vectorization through Hybrid Slice Selection. In *54th International Conference on Parallel Processing Companion (ICPP Companion '25)*, September 08–11, 2025, San Diego, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3750720.3757288>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICPP Companion '25, San Diego, CA, USA*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2109-0/25/09
<https://doi.org/10.1145/3750720.3757288>

'25), September 08–11, 2025, San Diego, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3750720.3757288>

1 Introduction

Modern processors are equipped with SIMD execution units, delivering a substantial boost in performance and energy efficiency when software can provide a stream of operations on independent data items. Hand-written intrinsics remain indispensable for a few critical libraries, but for the vast majority of applications the burden of exposing this data-level parallelism is carried by the compiler through automatic vectorization, the analysis and transformation that converts scalar code into SIMD instructions transparently to the programmer. This technique has become a standard feature of production tool-chains for CPUs and GPUs alike, and is now regarded as a cornerstone of high-performance code generation.

Automatic vectorizers in contemporary compilers, such as GCC[3] and LLVM[11], are organized around two complementary passes, Loop Vectorizer and SLP Vectorizer. The Loop Vectorizer[8][9] widens iterations of well-behaved loops whose induction variables, memory accesses and dependences can be proven statically. The intellectual foundations of loop vectorization trace back to the pioneering work of Allen and Kennedy in the late 1980s[6], who formalised dependence tests that allow a compiler to rewrite FORTRAN DO loops into vector form suitable for machines such as the Cray-1. While loop vectorization thrives on statically analyzable loops, it turns out overly restrictive for many real-world workloads with complex control flow.

Superword-Level Parallelism (SLP) [10], originally proposed by Larsen and Amarasinghe in 2000, was designed to automatically expose data-level parallelism in multimedia and scientific workloads. When combined with loop unrolling, SLP can uncover vectorization opportunities that conventional loop vectorizers overlook in the presence of intricate control-flow patterns. The SLP pass[15] begins by examining straight-line code, iteratively aggregating independent, isomorphic scalar instructions into vector packs and propagating these packs along use–definition chains. The outcome is an SLP tree whose nodes denote prospective vector operations. A target-dependent cost model then decides whether each node

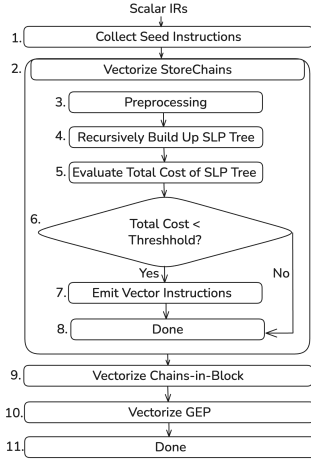


Figure 1: Overview of SLP Algorithm.

should be materialized as SIMD instructions, weighing the execution speed-up against the overhead of data gathering and scattering. If the projected gain exceeds a predefined profitability threshold, the tree is vectorized; otherwise, the packed instructions revert to their original scalar form. Importantly, once the pass elects to vectorize a given tree it commits to that decision greedily, without re-evaluating alternative packings. This myopic strategy can be suboptimal when reordering vectorization choices would yield a lower overall cost.

To mitigate the limitations outlined above, we introduce a hybrid slice selection algorithm that combines dynamic programming with a greedy heuristic. The dynamic-programming component is capable of minimizing the aggregate cost of multiple SLP trees, yielding an optimal selection when no inter-tree dependences are present. When such dependences do arise, the algorithm switches to a greedy strategy that iteratively chooses the tree with the lowest current cost, thereby targeting the vectorization opportunity with the greatest immediate performance benefit. Although this hybrid approach does not guarantee a globally optimal sequence of vectorization decisions, empirical evaluation shows that it achieves substantially larger overall cost reductions than the existing workflow.

2 Background and Motivation

This section begins with a concise overview of the Superword-Level Parallelism (SLP) algorithm. We then examine the preprocessing stage, highlighting specific areas in which the current approach can be refined. A detailed description of the existing algorithm follows, after which we present a motivating example that illustrates circumstances under which the existing method remains sub-optimal.

2.1 SLP Vectorization

Figure 1 outlines the Superword-Level Parallelism (SLP) pass. Its operation can be partitioned into three principal phases—Vectorize StoreChains (Step 2), Vectorize Chains-in-Block (Step 9), and Vectorize GEP (Step 10). In this paper, we primarily focus on the Vectorize

Storechains phase. The pass commences by collecting seed instructions (Step 1) from scalar intermediate representations (IRs). During this scan it identifies vectorization candidates that are either (i) store instructions or (ii) getelementptr (GEP) instructions. The resulting seeds are then forwarded to Vectorize StoreChains, where each store serves as the root of a candidate SLP tree that is constructed in a bottom-up fashion. Within Vectorize Store Chains the compiler first performs a pre-processing stage (Step 3). Stores are partitioned according to several correctness and profitability criteria—including independence of memory accesses, adjacency of target addresses, and uniformity of element type. This classification simultaneously preserves program semantics and yields a set of well-formed, high-potential seed groups for subsequent tree construction (Step 4) and cost evaluation (Step 5).

After the pre-processing step, each group of stores becomes the root of an SLP tree. The vectoriser then traverses the use-def chains bottom-up (Step 4), recursively adding scalar operations that compute the values eventually written by these stores. The recursion terminates when it encounters load or constant instructions, at which point the structure of the SLP tree is fully determined. For every candidate tree the pass performs a static profitability analysis (Step 5) driven by the target-specific cost model exposed through LLVM’s Target-Transform-Info (TTI) interface. At each node the *EntryCost*, the incremental cost (or saving) of replacing the scalar operations with their vector equivalent, is computed as

$$\text{EntryCost} = \text{VectorCost} - \text{ScalarCost}. \quad (1)$$

A negative value indicates that the vector form is cheaper than its scalar counterpart. Because vector values may later need to be extracted or inserted, the model augments the *EntryCost* with an *ExtractCost*, capturing the overhead of lane insertion or extraction. The cost attributed to an individual node therefore becomes

$$\text{NodeCost} = \text{EntryCost} + \text{ExtractCost}. \quad (2)$$

The total cost of the candidate tree is the sum of the *NodeCosts* of all nodes. The vectorizer then compares *TotalCost* with a predefined threshold (Step 6), typically set to 0. If *TotalCost* < threshold, the transformation is deemed profitable: the pass emits vector instructions for the entire tree (Step 7) and marks the region as done (Step 8). Otherwise, the scalar form is retained and the algorithm proceeds to the next candidate without modification.

$$\text{TotalCost} = \sum_{i=1}^{\text{size}(\text{SLPTree})} \text{NodeCost}(i) \quad (3)$$

2.2 Preprocessing of Vectorize StoreChains

Pre-processing constitutes a critical stage of LLVM’s SLP pass because it both delimits the region over which each SLP tree is formed and safeguards program correctness. Although some SLP variants allow seed collection across multiple basic blocks [12], the upstream LLVM implementation deliberately confines vectorization to a single basic block at a time. This design choice eliminates the need to reason about inter-block control flow, thereby reducing compilation overhead, yet it inevitably constrains the set of exploitable vectorization opportunities.

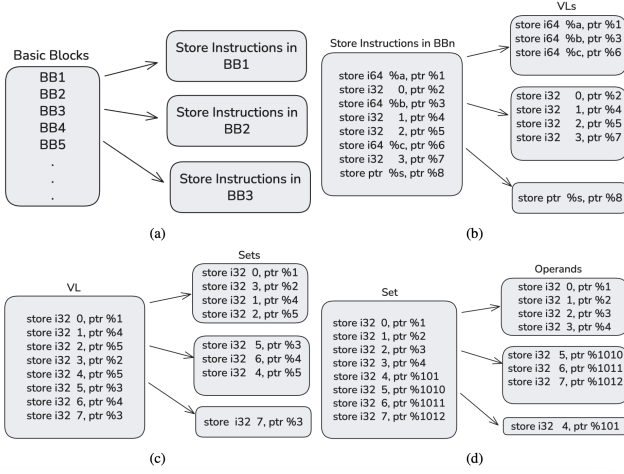


Figure 2: Preprocessing in Vectorize StoreChains

Figure 2(a) illustrates the first pre-processing step in Vectorize StoreChains: for every basic block, all store instructions are gathered into a local work-list. These stores are then partitioned into distinct vector lanes (VLs) according to the type of the value being written, as shown in Figure 2(b). In the example, 64-bit stores are grouped into the first VL, 32-bit stores into the second, and the lone pointer store into a third VL. This type-based stratification ensures that subsequent tree construction operates on homogeneous elements, which simplifies cost modelling and prevents illegal mixes of data widths.

Within each VL, the pass performs a refinement step that partitions the stores into sets that are provably free of write-after-write (WAW) hazards. The vectorizer processes the lane in program order and starts a fresh set whenever it encounters a store whose destination address has already appeared in the current set. This simple rule guarantees that no two stores in the same set may write to the same location, thereby preserving memory-dependence semantics. Once the membership of a set is fixed, its stores are reordered by ascending address so that the eventual vector write touches a contiguous (or at least monotonically increasing) region of memory, an arrangement that both satisfies hardware coalescing requirements and improves cache locality. In Figure 2(c) the first four writes to %1, %2, %4, and %5 populate Set 0; a second write to %5 triggers the creation of Set 1, which contains the next three stores; the final isolated write forms a singleton Set 2. The resulting sets are therefore WAW-safe and address-ordered, providing well-behaved building blocks for subsequent SLPTree construction.

The last pre-processing step subdivides each WAW-safe set into operands, grouping stores whose destination addresses are adjacent in memory. Because the stores in a set have already been ordered by address, adjacency can be detected with a single linear scan. In Figure 2(d) the first four stores target consecutive addresses and therefore form an Operand. The fifth write is separated by a gap, so it becomes the sole member of another Operand. The final three writes once again access adjacent locations and are collected into the other Operand. Each operand thus represents a maximally contiguous segment of the address space, and it is precisely at this

Algorithm 1: Existing Heuristic Slice Selection Algorithm

```

1 while true do
2   foreach VF in CandidateVFs do
3     startIdx ← 0;
4     while startIdx + VF ≤ size(Operand) do
5       slice ← Operand[startIdx : startIdx + VF - 1];
6       SLPTree ← buildTree(slice);
7       Cost ← SLPTree.cost;
8       if Cost < Threshold then
9         vectorize(SLPTree);
10        startIdx ← startIdx + VF;
11      else
12        startIdx ← startIdx + 1;
13      end
14    end
15  end
16 if doubling VF is valid then
17   CandidateVFs.append(2 × VFmax)
18   Continue;
19 end
20 if Operand cannot be further vectorized then
21   Break;
22 end
23 end

```

Figure 3: Existing Heuristic Slice Selection Algorithm

Operand	Slices	Cost
{A, B, C, D, E, F, G, H}	Slice1: {A, B, C, D}	−4
	Slice2: {E, F, G, H}	−4
	Slice3: {A, B}	−2
	Slice4: {C, D, E, F}	−5
	Slice5: {G, H}	−2

Table 1: Motivating Example

granularity that the SLP pass constructs candidate trees. Our work also focuses on enhancing the profitability analysis within this operand-level scope.

2.3 Existing Slice Selection Algorithm

As shown in Figure 3, Existing Slice Selection Algorithm adopts a greedy slice-selection strategy that iterates over a list of candidate vectorization factors (VFs). The VF_{max} is computed as the ratio of vector register width and element width, while the lower bound of VF is conventionally fixed at 2. Processing VFs from largest to smallest maximizes the chance of exploiting the full width of the target vector registers. For a given VF, the vectorizer scans the current Operand sequentially, extracting the next VF contiguous scalar instructions to form a slice. It then constructs an SLP tree for this slice and queries the target cost model to get the corresponding cost. If the resulting cost is profitable, i.e., $Cost < Threshold$, the slice is vectorized and its constituent instructions are removed from the operand to prevent re-visitation and the window slides forward by VF instructions. Otherwise the window slides forward by one instruction and the test repeats. When all candidate VFs have been examined, the algorithm checks whether the hardware permits doubling the largest VF; if so, it appends the doubled VF_{max} to the candidate list and restarts the search. This process continues until either the Operand is fully exhausted or no valid VF remains, at which point the slice-selection phase concludes.

Algorithm 2: Dynamic-programming Slice Selection Algorithm

```

1 cost_table  $\leftarrow N \times N$  table of costs, initialized to 0; //  $N = \text{size}(\text{Operands})$ 
2 DP  $\leftarrow$  array of  $N$  costs, initialized to 0;
3 Combination  $\leftarrow$  array that keeps pairs  $\langle \text{startIdx}, \text{endIdx} \rangle$ ;
4 if doubling VF is valid then
5   candidateVFs.append( $2 \times VF_{\max}$ )
6 end
7 // Fill in the cost_table
8 foreach VF in candidateVFs do
9   for startIdx  $\leftarrow 0$  to  $N - VF$  do
10    Slice  $\leftarrow \text{Operands}[\text{startIdx} : \text{startIdx} + VF - 1]$ ;
11    if Slice is profitable then
12      cost_table $[\text{startIdx}][\text{startIdx} + VF - 1] \leftarrow \text{Slice.get.cost}()$ ;
13    end
14  end
15 end
16 // Find Optimal combination of slices
17 for i  $\leftarrow 1$  to  $N - 1$  do
18   optimal_cost  $\leftarrow 0$ ;
19   optimal_VF  $\leftarrow 0$ ;
20   foreach VF in candidateVFs do
21     cost  $\leftarrow DP[i - VF] + \text{cost\_table}[i - VF + 1][i]$ ;
22     if cost < optimal_cost then
23       optimal_cost  $\leftarrow \text{cost}$ ;
24       optimal_VF  $\leftarrow VF$ ;
25     end
26   end
27   if optimal_cost  $\leq DP[i - 1]$  then
28     DP $[i] \leftarrow \text{optimal\_cost}$ ;
29     Combination $[i] \leftarrow \text{Combination}[i - \text{optimal\_VF}]$ ;
30     Combination $[i].\text{append}(\{i - \text{optimal\_VF} + 1, i\})$ ;
31   else
32     DP $[i] \leftarrow DP[i - 1]$ ;
33     Combination $[i] \leftarrow \text{Combination}[i - 1]$ ;
34   end
35 end

```

Figure 4: Dynamic Programming Slice Selection Algorithm

Nonetheless, Algorithm 1 may produce sub-optimal results in some cases. Consider an operand that comprises eight seed instructions $\{A, B, C, D, E, F, G, H\}$ whose slice costs are listed in Table 1. The heuristic begins with the maximum vectorization factor, $VF = 4$. It first selects slice 1 ($A-D$) because the estimated cost is negative, vectorizes it, and then proceeds to slice 2 ($E-H$), which is likewise profitable. The total saving realized for the operand is therefore -8 . A superior partition, however, is readily available: slice 3, slice 4, and slice 5, whose combined cost is -9 . The current SLP implementation fails to discover this arrangement because it evaluates slices greedily and in isolation, without a global view of the operand. Developing a slice-selection strategy that reasons holistically about multiple slices could therefore expose additional vectorization opportunities.

3 Method

We compute the cost of every admissible slice and frame slice selection as an optimization problem. When slices are independent, dynamic-programming (DP)[7] yields the minimum-cost tiling. If data dependences invalidate pure DP, we switch to a hybrid approach: greedy vectorizes the slice with minimum cost in the constrained region, and DP resumes once independence is restored. This preserves optimality when possible while remaining robust to dependences.

Algorithm 3: Hybrid Slice Selection Algorithm

```

1 while Profitable slice exists do
2   Find the optimal combination of slices using DP (Algorithm2);
3   if Slices are independent then
4     Vectorize the selected slices returned by DP;
5     Break;
6   else
7     Vectorize the slice with minimum cost;
8   end
9 end

```

Figure 5: Hybrid Slice Selection Algorithm

3.1 Dynamic Programming

As illustrated in Figure 4, algorithm 2 first extends the search space by provisionally doubling $VF_{\max}$. This permits wider slices to participate in the optimal tiling. Then it enumerates every admissible slice, queries the target cost model, and records the result in a two-dimensional array, *cost_table*. Profitable slices store their negative cost, whereas unprofitable ones are written as 0, signalling that they cannot appear in the final solution. The row and column indices of *cost_table* $[i][j]$ identify the first and last instructions of the slice within the operand. For instance, *cost_table* $[0][3]$ holds the cost of vectorizing the SLP tree that contains the first through the fourth seed instructions.

Once *cost_table* has been populated, a dynamic-programming (DP) stage selects the slice combination that minimizes total cost. A table entry *cost_table* $[i][j]$ is considered valid only when the bounds satisfy $0 \leq i < j < \text{size}(\text{Operand})$; for any out-of-range pair we assign a sentinel value of 0, ensuring that infeasible slices never take part in the optimisation:

$$\text{cost_table}[i][j] = 0, \quad \text{if } i \geq j \text{ or } i < 0 \text{ or } j \geq \text{size}(\text{Operand}).$$

Let $DP[n]$ denote the minimum attainable cost when only the first $n+1$ seed instructions (indices 0 through n) are eligible. Because a single instruction cannot be vectorized, the base case is

$$DP[n] = 0 \quad \text{for } n \leq 0.$$

For $n > 0$ the recurrence examines every candidate vectorization factor VF . It either (i) appends a profitable slice of width VF —whose cost is recorded in *cost_table* $[n - VF + 1][n]$ —to the best solution of the preceding prefix, or (ii) skips the current instruction altogether. Formally,

$$DP[n] = \min \left(\min_{VF \in \text{candidateVFs}} (DP[n - VF] + \text{cost_table}[n - VF + 1][n]), DP[n - 1] \right).$$

The lower portion of Algorithm 2 materializes the DP recurrence described above. A companion container, *Combination*, records the slice boundaries that realize the running optimum. After the DP table is filled for $n = 0$ through $\text{size}(\text{Operand}) - 1$, the final entry holds the minimum achievable cost, and *Combination* encodes the corresponding tiling of the operand. The vectorizer then emits vector code for each slice stored in *Combination*, thereby materializing the computed cost reduction.

Operand	Slices	Cost
{0, 1, 2, 3, 4, 5, 6, 7}	Slice1: {0, 1, 2, 3}	-8
	Slice2: {4, 5, 6, 7}	-8
	Slice3: {4, 5}	-3
	Slice4: {5, 6}	-3
	Slice5: {6, 7}	-3

Table 2: Case Study for Hybrid Algorithm

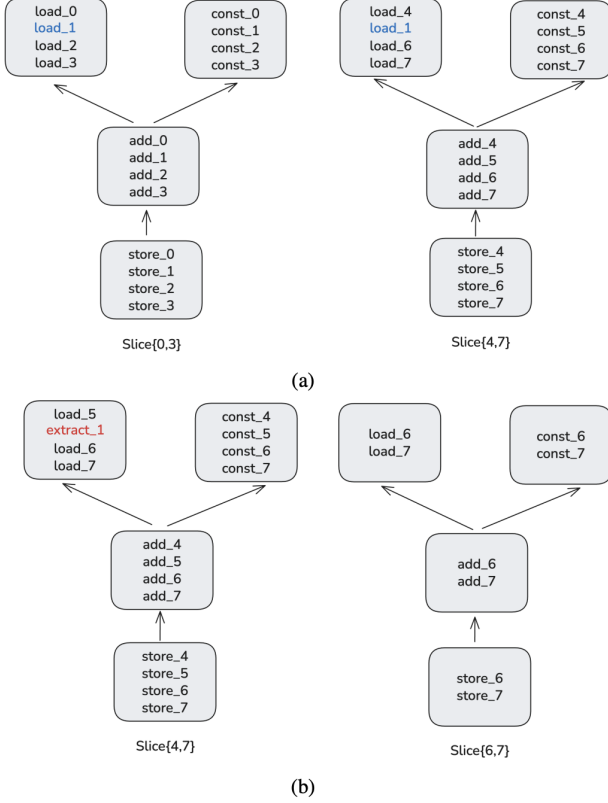


Figure 6: SLP Tree Demo for Hybrid Algorithm

3.2 Hybrid

Although DP can identify a static tiling with the greatest nominal cost reduction, such as the motivating case in Table 1, it breaks down when candidate SLP trees are inter-dependent. Consider the operand in Table 2, which contains eight store instructions. Pre-analysis assigns each admissible slice a cost before any vectorization is applied. Figure 6(a) shows the SLP trees for the two most lucrative slices, {0, 3} and {4, 7}; they share a common load (highlighted in blue), creating a dependence between the trees.

Because Algorithm 2 is oblivious to such interactions, it selects both slices, anticipating a total saving of -16. In the actual vectorization, slice {0, 3} is processed first and its transformation folds the shared load into the produced vector. When slice {4, 7} is examined next, that same load has become a vector extract (Figure 6(b)), introducing heterogeneity that the cost model penalizes heavily.

The slice is now unprofitable and is skipped, leaving an observed saving of only -8, far short of the -16 predicted by the DP stage.

To address inter-slice dependences we introduce the Hybrid Slice Selection Algorithm (Figure 5). Each iteration recomputes *cost_table*, applies dynamic programming to derive a provisional optimum, and simultaneously records dependence metadata while the table is populated. Before committing to the DP-selected tiling, the algorithm checks whether any of the chosen slices share a data dependence through data structure *ExternalUses* in LLVM. If none do, the slices are vectorized immediately, delivering the full cost reduction predicted by DP. If dependences are detected, the DP solution is discarded since the DP optimal-substructure property no longer holds and the algorithm falls back to a refined greedy step that vectorizes the single most profitable slice available at that moment. Although this greedy action forfeits global optimality guarantees, it ensures progress, tends to maximize data reuse in the remaining operand. The procedure iterates until either (i) a dependence-free DP tiling is found or (ii) no profitable slices remain. In pathological cases the greedy step may still perturb subsequent trees just like the scenario shown in Figure 6(b), but empirical evaluation shows that, for workloads amenable to SLP vectorization, the hybrid strategy usually surpasses the baseline heuristic in realized cost savings.

4 Experimental Setup

We incorporated the proposed Hybrid Slice-Selection algorithm into the upstream LLVM 19.1.0-rc3 compiler as an extension to the existing SLP pass. The implementation relies on the unmodified TargetTransformInfo (TTI)[4] interface for all cost queries, thereby using exactly the same cost model as the baseline pass.

Evaluation was carried out on kernels extracted from a selection of SPEC CPU 2006 and SPEC CPU 2017[5] C/C++ benchmarks. Experiments measured both static cost reduction, computed by the SLP pass, and dynamic instruction count, obtained from the execution on QEMU. Two vector-capable back ends were considered:

- **RISC-V RV64GCV (RVV[1])**
 - mcpu=generic-rv64
 - march=rv64gcv_zba_zbb_zbs
 - O3 -fno-vectorize -fslp-vectorize
 - mrvv-vector-bits=128
 - mllvm -slp-max-reg-size=128
 - mllvm -slp-min-reg-size=128
- **ARMv8 (Neon[2])**
 - march=armv8-a
 - O3 -fno-vectorize -fslp-vectorize
 - mllvm -slp-max-reg-size=128
 - mllvm -slp-min-reg-size=128

These settings disable the loop vectorizer (-fno-vectorize) to isolate the effects of the SLP pass, while constraining register sizes to 128 bits for a fair cross-architecture comparison.

5 Experimental Result

5.1 Static Cost Reduction

For the static-cost experiments we evaluate two configurations of the SLP pass: with and without VF-doubling. Doubling the maximum vectorization factor can uncover additional slicing opportunities, yet it also lengthens the produced vectors and thereby raises register pressure—an effect that LLVM’s current cost model does not capture [16]. On RISC-V, oversized vectors are accommodated at code-generation time by selecting a larger LMUL value, which aliases multiple physical vector registers into a single logical register and thus honors the width assumed during slicing. In contrast, Arm Neon provides no equivalent register grouping. The backend must therefore fracture an over-wide vector into several smaller registers.

Tables 3 through 10 compare the Hybrid Slice Selection algorithm with the original heuristic (Base) under two configurations—*with* and *without* VF-doubling. We report both the absolute change in static SLP cost and the corresponding percentage improvement.

- **With VF-doubling** the hybrid pass achieves substantially larger savings mostly on every benchmark suite.
 - On RISC-V the additional cost reduction reaches **67 %** (Table 3).
 - On Armv8 the peak improvement is **41 %** (Table 5).
- **Without VF-doubling** the gains are usually smaller because fewer slicing opportunities are exposed. Notable exceptions are 433.milc, 500.perlbench_r, and 508.namd_r on RISC-V, and 433.milc and 453.povray on Armv8, where the non-doubling configuration performs better. These outliers highlight a limitation of the greedy fallback in hybrid algorithm: when VF-doubling is enabled, it may vectorize a very wide slice that happens to be cheapest at that moment, screwing up the profitability of the remaining candidate trees. The resulting net improvement over the baseline therefore diminishes for those workloads.

Overall, enabling VF-doubling in conjunction with the hybrid selector yields the largest static cost reductions across both architectures and both SPEC suites.

5.2 Dynamic Instruction Count

Dynamic performance was measured under QEMU, but only for those benchmarks in which the hybrid selector achieved a lower *static* SLP cost than the baseline. This choice reflects the cost model’s assumption that a static saving should correlate with reduced run-time work. Each benchmark is reported as the ratio of dynamic instruction count relative to the original pass, using two compiler configurations:

- **With VF-doubling**
- **Without VF-doubling**

to assess whether longer vectors help or hinder performance.

Empirically, the dynamic counts seldom improve as dramatically as the static costs; in most cases they are statistically unchanged. Two possible factors are:

- (1) **Cost-model limitations.** LLVM’s current model does not precisely estimate the overhead of each instruction [13].

- (2) **Limited coverage.** The code regions transformed by the SLP pass only constitute only a minor share of total execution time[14]. Also, they are not on the run-time hotspots.

Despite this, the hybrid algorithm remains *portable*: as cost models become more accurate, the static reductions it realizes should manifest more strongly in run-time instruction counts.

6 Conclusion

This paper introduced a hybrid slice selection algorithm that augments LLVM’s SLP pass with a synergy of dynamic programming and refined greedy heuristics. By reasoning about slice order and explicitly tracking inter-tree dependences, the algorithm overcomes the myopia of the baseline heuristic. Comprehensive evaluation on SPEC CPU 2006 and 2017 kernels confirms the benefits of the approach on two disparate architectures. Relative to the existing LLVM implementation, the hybrid selector achieves up to 67 % additional static cost reduction on RISC-V and 41 % on Arm v8, while maintaining full compatibility with the existing TargetTransformInfo cost model. Although run-time instruction counts did not always mirror the static gains the method is inherently portable. As cost models evolve to be more accurate, we expect the hybrid scheme’s static savings to translate more directly into end-to-end performance.

7 Future Work

Although the hybrid slice selection algorithm markedly enlarges the static cost reduction, it does not yet reach the theoretical optimum. Establishing an upper bound on the achievable saving would require an exhaustive enumeration of all slice combinations—an investigation that could be prototyped with LLVM’s Sandbox IR infrastructure. Once this bound is known, attention can shift to refining the fallback greedy step that is invoked when inter-tree dependences arise. In particular, the heuristic should favour slices whose vectorized form maximizes data reuse in the remaining trees, rather than introducing heterogeneity that degrades subsequent opportunities.

Acknowledgments

This work was supported by the MediaTek Advanced Research Center (MRAC). Special appreciation is extended to thank Dr. Chi-Bang Kuan for his invaluable advice and Dr. Shih-wei Liao for fostering an engaging and supportive research environment.

References

- [1] 2021. *RISC-V Vector Extension*. <https://github.com/riscvarchive/riscv-v-spec>
- [2] 2025. *ARM NEON*. <https://www.arm.com/products/>
- [3] 2025. *Auto-vectorization in GCC*. <https://gcc.gnu.org/projects/tree-ssa/vectorization.html>
- [4] 2025. *llvm::TargetTransformInfo Class Reference*. https://llvm.org/doxygen/classllvm_1_1TargetTransformInfo.html
- [5] 2025. *Standard Performance Evaluation Corp Benchmarks*. <https://www.spec.org/>
- [6] Randy Allen and Ken Kennedy. 1987. Automatic Translation of FORTRAN Programs to Vector Form. In *ACM Transactions on Programming Languages and Systems* (1987). 491–542.
- [7] Richard Bellman. 1966. Dynamic Programming. In *Science*. 34–37.
- [8] Nuzman Dorit and Zaks Ayal. 2008. Outer-loop vectorization: revisited for short SIMD architectures. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*. 2–11.

- [9] Nuzman Dorit, Rosen Ira, and Zaks Ayal. 2006. Auto-vectorization of interleaved data for SIMD. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 132–143.
- [10] Samuel Larsen and Saman Amarasinghe. 2000. Exploiting superword level parallelism with multimedia instruction sets. In *Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation(PLDI)*. 145–156.
- [11] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization(CGO)*. 75–86.
- [12] Yishen Chen, Charith Mendis and Saman Amarasinghe. 2022. All You Need Is Superword-Level Parallelism: Systematic Control-Flow Vectorization with SLP. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation(PLDI)*. 301–315.
- [13] Angela Pohl, Biagio Cosenza, and Ben Juurlink. 2019. Portable Cost Modeling for Auto-Vectorizers. In *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems(MASCOTS)*. 359–369.
- [14] Vasileios Porpodas and Timothy M. Jones. 2015. Throttling Automatic Vectorization: When Less is More. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 432–444.
- [15] I. Rosen, D. Nuzman, and A. Zaks. 2007. Loop-aware SLP in GCC. In *GCC Developers Summit*.
- [16] Meng-Shiuan Shih, Hung-Ming Lai, Chao-Lin Lee, Chung-Kai Chen, and Jenq-Kuen Lee. 2023. Register-Pressure Aware Predicator for Length Multiplier of RVV. In *ICPP Workshops '22: Workshop Proceedings of the 51st International Conference on Parallel Processing*. 1–9.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
400.perlbench	–	–119	–119	–
401.bzip	–	–8	–8	–
403.gcc	–69 (–11%)	–619	–688	-1.03×10^{-4}
429.mcf	–	–2	–2	–
445.gobmk	–60 (–48%)	–126	–186	-8.71×10^{-7}
456.hmmer	–6 (–16%)	–37	–43	-8.45×10^{-11}
458.sjeng	–12 (–67%)	–18	–30	-5.32×10^{-4}
462.libquantum	–	–18	–18	–
464.h264ref	–148 (14%)	–1049	–1197	-3.61×10^{-6}
471.omnetpp	–	–8	–8	–
473.astar	–	–15	–15	–
433.milc	19 (+12%)	–163	–144	$+2.26 \times 10^{-4}$
444.namd	–	–6	–6	–
447.dealII	–3 (–3%)	–106	–109	$+5.66 \times 10^{-11}$
450.soplex	–1 (–10%)	–10	–11	$+2.77 \times 10^{-11}$
453.povray	–21 (–3%)	–709	–730	$+7.04 \times 10^{-5}$
470.lbm	–1 (–33%)	–2	–3	$+1.39 \times 10^{-5}$
482.sphinx3	–	–28	–28	–

Table 3: SPEC CPU 2006 results with doubleVF on RISC-V.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
400.perlbench	–	–119	–119	–
401.bzip	–	–8	–8	–
403.gcc	–	–586	–586	–
429.mcf	–	–2	–2	–
445.gobmk	–20 (–16%)	–125	–145	-3.60×10^{-7}
456.hmmer	–	–28	–28	–
458.sjeng	–	–18	–18	–
462.libquantum	–	–16	–16	–
464.h264ref	–40 (–4%)	–1019	–1059	-2.01×10^{-10}
471.omnetpp	–	0	0	–
473.astar	–	–15	–15	–
433.milc	–51 (–52%)	–98	–149	-1.25×10^{-4}
444.namd	–	–6	–6	–
447.dealII	–	–102	–102	–
450.soplex	–1 (–10%)	–10	–11	$+3.30 \times 10^{-10}$
453.povray	–16 (–2%)	–664	–680	$+7.03 \times 10^{-5}$
470.lbm	–	–2	–2	–
482.sphinx3	–	–28	–28	–

Table 4: SPEC CPU 2006 results without doubleVF on RISC-V.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
400.perlbench	–	–138	–138	–
401.bzip	–	–40	–40	–
403.gcc	–41 (–4%)	–946	–987	-1.45×10^{-3}
429.mcf	–	–4	–4	–
445.gobmk	–58 (–41%)	–141	–199	-3.21×10^{-7}
456.hmmer	–8 (–8%)	–139	–147	$+3.14 \times 10^{-11}$
458.sjeng	–	–61	–61	–
462.libquantum	–	–6	–6	–
464.h264ref	–134 (–9%)	–1519	–1653	$+4.05 \times 10^{-6}$
471.omnetpp	–	–33	–33	–
473.astar	–	–17	–17	–
433.milc	–36 (–9%)	–386	–422	-1.70×10^{-5}
444.namd	–72 (–29%)	–247	–319	-1.35×10^{-2}
447.dealII	–2 (–1%)	–278	–280	0
450.soplex	–	–41	–41	–
453.povray	2 (+0%)	–1012	–1010	-8.84×10^{-7}
470.lbm	–7 (–24%)	–29	–36	-2.45×10^{-9}
482.sphinx3	–	–58	–58	–

Table 5: SPEC CPU 2006 results with doubleVF on ARMv8.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
400.perlbench	–	–138	–138	–
401.bzip	–	–40	–40	–
403.gcc	–8 (–1%)	–946	–954	-6.54×10^{-4}
429.mcf	–	–4	–4	–
445.gobmk	–30 (–21%)	–141	–171	-1.29×10^{-7}
456.hmmer	–4 (–3%)	–139	–143	$+2.55 \times 10^{-11}$
458.sjeng	–	–61	–61	–
462.libquantum	–	–6	–6	–
464.h264ref	–115 (–8%)	–1464	–1579	$+7.36 \times 10^{-6}$
471.omnetpp	–	–33	–33	–
473.astar	–	–17	–17	–
433.milc	–58 (–18%)	–320	–378	-2.04×10^{-4}
444.namd	–	–66	–66	–
447.dealII	–2 (–1%)	–278	–280	0
450.soplex	–	–41	–41	–
453.povray	–5 (0%)	–976	–981	-2.88×10^{-7}
470.lbm	–3 (–10%)	–29	–32	-1.94×10^{-11}
482.sphinx3	–	–58	–58	–

Table 6: SPEC CPU 2006 results without doubleVF on ARMv8.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
500.perlbench_r	+1 (+3%)	-35	-34	$+1.39 \times 10^{-6}$
502.gcc_r	-50 (-13%)	-374	-424	-6.86×10^{-5}
505.mcf_r	-	0	0	-
520.omnetpp_r	-	-20	-20	-
525.x264_r	-84 (-14%)	-569	-653	-3.00×10^{-5}
531.deepsjeng_r	-	-75	-75	-
541.leela_r	-	-6	-6	-
557.xz_r	-34 (-7%)	-507	-541	-7.35×10^{-10}
508.namd_r	-75 (-5%)	-1 626	-1 701	$+7.78 \times 10^{-5}$
510.parest_r	-	-375	-375	-
511.povray_r	-21 (-3%)	-705	-726	$+7.14 \times 10^{-5}$
519.lbm_r	-1 (-50%)	-2	-3	$+1.09 \times 10^{-5}$
538.imagick_r	-32 (-5%)	-678	-710	$+1.17 \times 10^{-10}$
544.nab_r	-4 (-4%)	-96	-100	$+2.13 \times 10^{-10}$

Table 7: SPEC CPU 2017 results with doubleVF on RISC-V.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
500.perlbench_r	-	-35	-35	-
502.gcc_r	-1 (0%)	-362	-363	$+1.38 \times 10^{-5}$
505.mcf_r	-	0	0	-
520.omnetpp_r	-	-20	-20	-
525.x264_r	-24 (-4%)	-560	-584	$+1.77 \times 10^{-6}$
531.deepsjeng_r	-	-75	-75	-
541.leela_r	-	-6	-6	-
557.xz_r	-	-485	-485	-
508.namd_r	-117 (-7%)	-1 626	-1 743	$+7.80 \times 10^{-5}$
510.parest_r	-	-363	-363	-
511.povray_r	-16 (-2%)	-660	-676	$+2.06 \times 10^{-4}$
519.lbm_r	-	-2	-2	-
538.imagick_r	-13 (-2%)	-666	-679	$+2.85 \times 10^{-10}$
544.nab_r	-	-76	-76	-

Table 8: SPEC CPU 2017 results *without* doubleVF on RISC-V.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
500.perlbench_r	+2 (+1%)	-134	-132	$+1.15 \times 10^{-8}$
502.gcc_r	-44 (-4%)	-1 149	-1 193	$+7.30 \times 10^{-5}$
505.mcf_r	-	0	0	-
520.omnetpp_r	-	-116	-116	-
525.x264_r	-113 (-8%)	-1 422	-1 535	$+3.14 \times 10^{-6}$
531.deepsjeng_r	-	-100	-100	-
541.leela_r	-	-23	-23	-
557.xz_r	-45 (-5%)	-936	-981	$+2.57 \times 10^{-10}$
508.namd_r	-211 (-30%)	-699	-910	$+9.93 \times 10^{-4}$
510.parest_r	-	-896	-896	-
511.povray_r	+2 (+0%)	-1 007	-1 005	$+1.16 \times 10^{-8}$
519.lbm_r	-7 (-24%)	-29	-36	$+9.70 \times 10^{-12}$
538.imagick_r	-42 (-4%)	-1 041	-1 083	$+1.55 \times 10^{-10}$
544.nab_r	-	-202	-202	-

Table 9: SPEC CPU 2017 results with doubleVF on ARMv8.

Benchmark	Cost Diff.	Base	Hybrid	Dyn. Instr. Cnt
500.perlbench_r	+2 (+1%)	-134	-132	$+2.09 \times 10^{-6}$
502.gcc_r	-1 (0%)	-1 148	-1 149	$+7.44 \times 10^{-5}$
505.mcf_r	-	0	0	-
520.omnetpp_r	-	-116	-116	-
525.x264_r	-95 (-7%)	-1 374	-1 469	$+3.17 \times 10^{-6}$
531.deepsjeng_r	-	-100	-100	-
541.leela_r	-	-23	-23	-
557.xz_r	-45 (-5%)	-933	-978	-3.53×10^{-10}
508.namd_r	-138 (-26%)	-541	-679	$+5.54 \times 10^{-4}$
510.parest_r	-	-896	-896	-
511.povray_r	-5 (0%)	-971	-976	-1.14×10^{-8}
519.lbm_r	-3 (-10%)	-29	-32	$+4.85 \times 10^{-12}$
538.imagick_r	-39 (-4%)	-1 027	-1 066	1.57×10^{-10}
544.nab_r	-	-202	-202	-

Table 10: SPEC CPU 2017 results *without* doubleVF on ARMv8.